

Query Request Body

Overview

Every key is optional except `resource`.

```
resource, where, expand, unwind, filter, over, select, sort, pagination
```

The query language is Mongo-flavored. If you've written MongoDB filters before, most of this will feel familiar.

Query Pipeline

```
where (filter raw records)
  ↓
expand (hydrate references)
  ↓
unwind (fan arrays into rows)
  ↓
filter (filter hydrated rows)
  ↓
over (partition into groups)
  ↓
select (paths + expand + reductions)
  ↓
sort (order results)
  ↓
paginate (limit/offset)
```

Omitted steps are no-ops.

Filtering

A `where` value mirrors the shape of your records. Keys are either field names or operators (starting with \$).

```
{ "resource": "Policy", "where": { "status": "active" } }
// where Policy.status == 'active'

{ "resource": "Location", "where": { "hazard_tags": ["hail", "wildfire"] } }
// where Location.hazard_tags === ['hail', 'wildfire']
```

To use operators:

```
{ "where": { "premium": { "$gte": 5000, "$lt": 20000 } } }
// where 5000 <= $.premium < 20000
```

Operators

Operator	Meaning	Works On
<code>\$eq</code>	Deep equality	Any type
<code>\$ne</code>	Not equal	Any type
<code>\$exists</code>	Field exists	Any type
<code>\$gt</code> , <code>\$gte</code> , <code>\$lt</code> , <code>\$lte</code>	Comparisons	Scalars
<code>\$in</code>	In list	Scalars/arrays
<code>\$nin</code>	Not in list	Scalars/arrays
<code>\$contains</code>	Contains substring/element	Scalars/arrays
<code>\$elemMatch</code>	At least one array element matches	Arrays only

Nested Objects

For nested objects, nesting and dot-paths are equivalent:

```
{ "where": { "dates": { "effective": "2026-01-01" } } }
{ "where": { "dates.effective": "2026-01-01" } }
```

Be careful with arrays: Use `$elemMatch` to break paths at array boundaries.

Array Clauses

`$elemMatch` passes if at least one element satisfies the sub-clause:

```
{
  "expand": { "exposure_units": { "location": true } },
  "filter": {
    "exposure_units": {
      "$elemMatch": {
        "kind": "location",
        "location": { "hazard_tags": { "$in": ["earthquake", "wildfire"] } }
      }
    }
  }
}
```

Combinators

`$and` , `$or` , `$not` work as expected. Multiple operators in one clause implicitly form `$and` .

```
{
  "where": {
    "status": "active",
    "$or": [
      { "business_type": "renewal" },
      { "premium": { "$gte": 100000 } }
    ],
    "$not": { "producer.broker": 2 }
  }
}
```

References

Some fields are references: they store IDs pointing to records in another resource.

Two ways to resolve:

```
{ "expand": { "producer": "broker" } }
// A STAGE. Hydrates the record, so filter/over/sort/select can reach through producer.broker.

{ "select": { "producer": { "broker": { "$expand": true } } } }
// A SELECT LEAF. Resolves in the output only.
```

Use `expand` when something downstream needs the reference. Use `$expand` when you only want it in the reply.

The expand Stage

```
{ "expand": { "producer": "broker" } }
{ "expand": { "producer": { "broker": true } } }
{ "expand": { "producer": { "broker": {} } } }
```

Chain by nesting:

```
{ "expand": { "exposure_units": { "location": { "buildings": true } } } }
```

The \$expand Leaf

```
{
  "select": {
    "policy_number": true,
    "exposure_units": {
      "$expand": {
        "select": {
          "kind": true,
          "underlying_layer": {
            "policy": { "$expand": { "select": ["policy_number", "limit"] } }
          }
        }
      }
    }
  }
}
```

Projection

Omit `select` to return the entire record shape.

Two forms:

```
{ "select": ["id", "status", "dates.effective"] }
{ "select": { "id": true, "status": true, "dates": { "effective": true } } }
```

A select leaf is one of three things:

```
{
  "select": {
    "policy_number": true,                                // field path
    "broker": { "$expand": { "select": ["name"] } },      // reference
    "totalTiv": { "$sum": "exposure_units.basis_amount" } // aggregation
  }
}
```

Aggregations

Function	Argument	Notes
<code>\$sum</code>	path	Non-numeric/null skipped. Sum of nothing = 0
<code>\$avg</code>	path	Nulls skipped. Average of nothing = null
<code>\$min</code> / <code>\$max</code>	path	Nulls skipped. Result over nothing = null
<code>\$count</code>	<code>true</code>	Counts every row, nulls included
<code>\$countDistinct</code>	path or array	Distinct over non-null only

Unwinding

Each entry is a path string or `{ path, type }`.

```
{ "unwind": [{ "path": "claims", "type": "inner" }] } // k records; k <= N
{ "unwind": [{ "path": "claims", "type": "left" }] } // N records
```

Fan-Out

Return a stream of associated objects with parent information:

```
{ "unwind": ["exposure_units"],
  "select": ["id", "policy_number", "exposure_units"] }
```

To recover all exposure units per policy, include an `over` clause:

```
{ "unwind": ["exposure_units"],
  "over": ["id", "exposure_units.id"],
  "select": ["id", "policy_number", "exposure_units"] }
```

Grouping

Much like SQL GROUP BY, `over` partitions rows. Defaults to `['id']`.

```
{ "over": ["line_of_business", "exposure_units.basis"] }
```

Groups come back as flat projections in first-seen order:

```
{
  "resource": "Policy",
  "total": 2,
  "groups": [
    { "line_of_business": "property", "policies": 27, "premium": 6690900 },
    { "line_of_business": "cgl", "policies": 11, "premium": 2193000 }
  ]
}
```

Ordering

```
{ "sort": [{ "field": "premium", "direction": "desc" }, { "field": "id" }] }
```

Rules apply in priority order. `direction` defaults to “asc”. Nulls sort last.

Sort runs after select, so it can reference derived paths:

```
{
  "over": ["producer.broker.id"],
  "select": {
    "producer": { "broker": { "name": true } },
    "totalPremium": { "$sum": "premium" }
  },
  "sort": [{ "field": "totalPremium", "direction": "desc" }],
  "pagination": { "limit": 5 }
}
```

Pagination

```
{ "pagination": { "limit": 10, "offset": 20 } }
```

The `total` in results reports total matching records, irrespective of pagination.

Worked Examples

Non-Expired Policies with Expanded Details

```
{
  "resource": "Policy",
  "where": { "status": { "$ne": "expired" } },
  "expand": { "insured": true },
  "filter": { "insured.naics_code": "541110" },
  "select": [
    "policy_number",
    "line_of_business",
    "premium",
    { "insured": ["name", "entity_type", "employee_count"] }
  ],
  "sort": [{ "field": "insured.name", "direction": "asc" }]
}
```

Auto Policies with Fleet Summarized

```

{
  "resource": "Policy",
  "where": { "line_of_business": "auto" },
  "expand": { "exposure_units": true },
  "unwind": ["exposure_units"],
  "filter": { "exposure_units.kind": "vehicle" },
  "select": {
    "policy_number": true,
    "premium": true,
    "vehicles": { "$count": true },
    "fleetCost": { "$sum": "exposure_units.vehicle.cost_new" },
    "oldestYear": { "$min": "exposure_units.vehicle.year" }
  },
  "sort": [{ "field": "fleetCost", "direction": "desc" }],
  "pagination": { "limit": 10 }
}

```

Pre-1990 Buildings at Catastrophe-Exposed Locations

```

{
  "resource": "Policy",
  "where": {
    "$and": [
      { "status": { "$in": ["active", "bound"] } },
      { "line_of_business": { "$in": ["property", "cgl"] } }
    ]
  },
  "expand": { "insured": true, "exposure_units": { "location": { "buildings": true } } },
  "unwind": ["exposure_units", "exposure_units.location.buildings"],
  "filter": {
    "$and": [
      { "exposure_units.location.hazard_tags": { "$in": ["hurricane", "flood", "earthquake", "wildfire"] } },
      { "exposure_units.location.buildings.sprinklered": false },
      { "exposure_units.location.buildings.year_built": { "$lte": 1990 } }
    ]
  },
  "over": ["exposure_units.location.state"],
  "select": {
    "exposure_units": { "location": { "state": true } },
    "accounts": { "$countDistinct": "insured.id" },
    "policies": { "$countDistinct": "id" },
    "buildings": { "$count": true },
    "exposed": {
      "tiv": { "$sum": "exposure_units.location.buildings.tiv" },
      "largest": { "$max": "exposure_units.location.buildings.tiv" }
    }
  },
  "sort": [{ "field": "exposed.tiv", "direction": "desc" }]
}

```